OnlineCourseCompare: Database Setup Guide

The reviews database: worth it or money grab, AI slop meter, tags and comments • version 2 • November 4, 2026

What this is

The website's courses are built into the website itself. This database only stores the anonymous reviews: "Did you take this course?", worth it or money grab, the AI slop score, how much people learned, tags, comments, "was this helpful" votes and reports. If it is ever paused or deleted, **the site keeps working**; only reviews stop until you set it up again.

The setup script is saved in the website code at `online-course-next/supabase/setup.sql` (and on GitHub), and printed in full at the end of this guide. Running it on any new, empty Supabase project rebuilds the same database in about a minute.

1. Create the project

1. Go to `supabase.com/dashboard`, open your **free** organization, and click **New project**.
2. Name: `onlinecoursecompare`. Database password: click **Generate** and save it in your password manager.
3. Region: **East US (North Virginia)**. The website runs in Vercel's Washington, D.C. region, so this keeps reviews fast.
4. Click **Create new project** and wait about 2 minutes for it to start.

2. Run the setup script

1. In the left menu, open **SQL Editor** and click **New query**.
2. Paste the whole script: copy it from `supabase/setup.sql` or from the last pages of this guide.
3. Click **Run**. You should see **Success. No rows returned**.
4. Check: open **Table Editor**. You should see 5 tables: `course_reviews`, `course_review_tags`, `review_helpful_votes`, `review_reports` and `review_tags` (the last one already has 16 tags in it).

Running the script again is safe: it only adds what is missing and never deletes reviews. If you ran version 1 before, it removes the old `course_votes` table if nobody voted in it. Supabase's Security Advisor may say **RLS Enabled No Policy** for these tables. That is on purpose: only the website's server is allowed to read or write reviews.

3. Connect the website

1. In Supabase, click **Connect** at the top of the project page (or go to **Project Settings > API Keys**). Copy the **Project URL** and a **Secret key** (it starts with `sb_secret_`; click **Create new API keys** if there isn't one).
2. In Vercel, open the `online-course-next` project, go to **Settings > Environment Variables**, and add:

```
SUPABASE_URL         = the Project URL
SUPABASE_SECRET_KEY  = the secret key
```

3 Redeploy: **Deployments** > the latest one > **...** > **Redeploy**.

Keep the secret key secret. Never paste it into code, GitHub, a chat, or a variable that starts with `NEXT_PUBLIC_`. If it leaks, delete it under **API Keys** and make a new one. The website code that uses these two variables is added in Phase 2, so this step only matters once that is live.

4. Keeping it free and safe

- **Pausing:** Supabase pauses free projects after about a week with almost no database activity, and emails you a warning first. Phase 2 adds a daily check from the website that keeps it active.
- **If it gets paused:** open the dashboard, click the project, then **Resume project**. This works for 90 days after the pause, with all reviews kept.
- **After 90 days:** you can still download a backup from the project page, or start fresh: make a new project and run the setup script again.
- **Back up the reviews** (once a month is plenty): in **Table Editor**, export `course_reviews` and `course_review_tags` as CSV (also `review_helpful_votes` and `review_reports` if you want those). Keep the files somewhere safe.

To restore reviews into a new project:

1 Create the project and run the setup script (steps 1 and 2).

2 In **Table Editor**, open `course_reviews`, choose **Insert > Import data from CSV**, and pick its backup file. Then do the same for `course_review_tags` and the others (reviews always go first).

3 In **SQL Editor**, run this line once so new reviews don’t clash with the restored ones:

```
select setval(pg_get_serial_sequence('public.course_reviews', 'id'),
              (select coalesce(max(id), 0) + 1 from public.course_reviews), false);
```

5. Moderating reviews

- **Automatic:** a review that 3 different visitors report is hidden right away. Editing it does not bring it back.
- **By hand:** in **Table Editor**, open `course_reviews` and change `status` to `hidden` (or back to `published` if the reports were unfair). Reports are listed in `review_reports`.
- **Tags:** the tag list is near the top of `setup.sql`. Edit the labels or add tags there and run the script again; reviews keep their tags.
- **Spam brake:** one visitor can write or edit at most 20 reviews a day.

6. Handing it to Claude (or anyone) later

Give them this guide and access to the GitHub repo, and say something like:

> *“Set up the OnlineCourseCompare reviews database: run online-course-next/supabase/setup.sql in my new Supabase project, then add SUPABASE_URL and SUPABASE_SECRET_KEY to the Vercel project and redeploy.”*

- If Claude has a Supabase connection to that organization, it can run the script for you. If not, you paste it into the SQL Editor yourself (step 2); it is the same result either way.
- **Moving it to another account:** Supabase can move a project between organizations (**Project Settings > General > Transfer project**). You need to be the owner of the organization it is in now, and a member of the one it is going to.

The setup script

Same as `online-course-next/supabase/setup.sql`. Copy everything in the grey box (it may continue on the next page).

```
-- ======================================================================
-- OnlineCourseCompare: database setup for course reviews
-- ("Worth it or money grab?", the AI slop meter, tags and comments)
--
-- Run this in your Supabase project:
--   Supabase dashboard -> your project -> SQL Editor -> New query
--   -> paste this whole file -> Run
--
-- Safe to run again: it only adds what is missing and refreshes the tag
-- list, views and functions. It never deletes reviews. Back up the
-- reviews themselves separately (see the setup guide PDF).
--
-- The course list is NOT in the database. It lives in the website
-- (src/data/courses.json), so the site keeps working even if this
-- database is paused or gone. Only reviews depend on it.
--
-- Nobody signs in. voter_hash is a one-way hash the website's server
-- makes from the visitor's connection: one review per visitor per course
-- (which they can edit), and no IP address is ever stored.
-- ======================================================================


-- Version 1 of this script made a simpler course_votes table. Remove it
-- if it is still empty; if anyone has voted in it, leave it alone.
do $$
begin
  if to_regclass('public.course_votes') is not null then
    if not exists (select 1 from public.course_votes) then
      drop view if exists public.course_vote_totals;
      drop view if exists public.platform_vote_totals;
      drop table public.course_votes;
    end if;
  end if;
end $$;


-- ----------------------------------------------------------------------
-- Reviews
-- ----------------------------------------------------------------------
create table if not exists public.course_reviews (
  id          bigint generated by default as identity primary key,
  course_id   bigint      not null,   -- "id" from src/data/courses.json
  platform    text        not null,   -- 'Udemy', 'Coursera', ...
  voter_hash  text        not null,

  -- "Did you take this course?" Only people who did can review it.
  took_course text        not null
    check (took_course in ('finished', 'in_progress', 'dropped')),

  -- The one-tap answer every review has.
  verdict     text        not null check (verdict in ('worth_it', 'money_grab')),

  -- Optional 1-5 scores.
  --   ai_slop:    1 = made by a real person, 5 = mostly AI filler
  --   learned:    1 = learned nothing,       5 = learned a lot
  --   difficulty: 1 = very easy,             5 = very hard
  ai_slop     smallint    check (ai_slop between 1 and 5),
  learned     smallint    check (learned between 1 and 5),
  difficulty  smallint    check (difficulty between 1 and 5),
  paid        text
    check (paid in ('free', 'discount', 'full_price', 'subscription')),
  comment     text        check (char_length(comment) between 10 and 2000),

  -- Hidden reviews are not shown anywhere. 3 reports hide one
  -- automatically; change it back in the Table Editor if it was fine.
  status      text        not null default 'published'
    check (status in ('published', 'hidden')),
```

```
  created_at  timestamptz not null default now(),
  updated_at  timestamptz not null default now(),
  constraint course_reviews_one_per_voter unique (course_id, voter_hash)
);

create index if not exists course_reviews_platform_idx
  on public.course_reviews (platform);
create index if not exists course_reviews_voter_recent_idx
  on public.course_reviews (voter_hash, updated_at);


-- ---------------------------------------------------------------------
-- Tags people can pick (up to 5 per review). Edit the list here and run
-- the script again to rename or add tags.
-- ---------------------------------------------------------------------
create table if not exists public.review_tags (
  code       text     primary key,
  label      text     not null,
  kind       text     not null check (kind in ('good', 'bad')),
  sort_order smallint not null default 0
);

insert into public.review_tags (code, label, kind, sort_order) values
  ('too_much_ai',        'Too much AI content',            'bad',   10),
  ('ai_voiceover',       'AI voice-over',                  'bad',   20),
  ('filler',             'Padded with filler',             'bad',   30),
  ('outdated',           'Outdated',                       'bad',   40),
  ('upsells',            'Constant upsells',               'bad',   50),
  ('misleading',         'Not what the description said',  'bad',   60),
  ('no_support',         'No help from the instructor',    'bad',   70),
  ('cert_useless',       'Certificate not worth it',       'bad',   80),
  ('clear_teaching',     'Clear explanations',             'good', 110),
  ('real_projects',      'Real projects',                  'good', 120),
  ('job_ready',          'Job-ready skills',               'good', 130),
  ('beginner_friendly',  'Beginner friendly',              'good', 140),
  ('up_to_date',         'Up to date',                     'good', 150),
  ('helpful_instructor', 'Instructor answers questions',   'good', 160),
  ('great_value',        'Great value',                    'good', 170),
  ('cert_useful',        'Certificate helped me',          'good', 180)
on conflict (code) do update
  set label = excluded.label, kind = excluded.kind, sort_order = excluded.sort_order;

create table if not exists public.course_review_tags (
  review_id bigint not null references public.course_reviews (id) on delete cascade,
  tag       text   not null references public.review_tags (code) on update cascade,
  primary key (review_id, tag)
);

create index if not exists course_review_tags_tag_idx
  on public.course_review_tags (tag);


-- ---------------------------------------------------------------------
-- "Was this review helpful?" and "Report this review"
-- ---------------------------------------------------------------------
create table if not exists public.review_helpful_votes (
  review_id  bigint      not null references public.course_reviews (id) on delete cascade,
  voter_hash text        not null,
  helpful    boolean     not null,
  created_at timestamptz not null default now(),
  primary key (review_id, voter_hash)
);

create table if not exists public.review_reports (
  review_id  bigint      not null references public.course_reviews (id) on delete cascade,
  voter_hash text        not null,
  reason     text        not null
    check (reason in ('spam', 'offensive', 'fake', 'off_topic', 'personal_info')),
  details    text        check (char_length(details) <= 500),
  created_at timestamptz not null default now(),
  primary key (review_id, voter_hash)
);

create or replace function public.hide_reported_review()
```

```
returns trigger
language plpgsql
set search_path = ''
as $$
begin
  if (select count(*) from public.review_reports where review_id = new.review_id) >= 3 then
    update public.course_reviews set status = 'hidden' where id = new.review_id;
  end if;
  return new;
end;
$$;

create or replace trigger review_reports_hide_after_3
  after insert on public.review_reports
  for each row execute function public.hide_reported_review();


-- ---------------------------------------------------------------------
-- Writing a review: one call saves the review and its tags together.
-- Sending the same voter_hash and course again edits the review.
-- ---------------------------------------------------------------------
create or replace function public.submit_review(
  p_course_id   bigint,
  p_platform    text,
  p_voter_hash  text,
  p_took_course text,
  p_verdict     text,
  p_ai_slop     integer default null,
  p_learned     integer default null,
  p_difficulty  integer default null,
  p_paid        text    default null,
  p_comment     text    default null,
  p_tags        text[]  default '{}'
)
returns bigint
language plpgsql
set search_path = ''
as $$
declare
  v_id bigint;
begin
  if coalesce(array_length(p_tags, 1), 0) > 5 then
    raise exception 'Pick at most 5 tags';
  end if;

  -- Spam brake: one visitor can write or edit at most 20 reviews a day.
  if (select count(*) from public.course_reviews
      where voter_hash = p_voter_hash
        and course_id <> p_course_id
        and updated_at > now() - interval '1 day') >= 20 then
    raise exception 'Too many reviews today';
  end if;

  insert into public.course_reviews as r
    (course_id, platform, voter_hash, took_course, verdict,
     ai_slop, learned, difficulty, paid, comment)
  values
    (p_course_id, p_platform, p_voter_hash, p_took_course, p_verdict,
     p_ai_slop, p_learned, p_difficulty, p_paid, nullif(btrim(p_comment), ''))
  on conflict on constraint course_reviews_one_per_voter do update set
    took_course = excluded.took_course,
    verdict     = excluded.verdict,
    ai_slop     = excluded.ai_slop,
    learned     = excluded.learned,
    difficulty  = excluded.difficulty,
    paid        = excluded.paid,
    comment     = excluded.comment,
    updated_at  = now()      -- status is kept: editing never un-hides a review
  returning r.id into v_id;

  delete from public.course_review_tags where review_id = v_id;
  insert into public.course_review_tags (review_id, tag)
    select distinct v_id, t from unnest(coalesce(p_tags, '{}')) as t;

  return v_id;
```

```
end;
$$;


-- ------------------------------------------------------------------
-- What the website shows. Only published reviews count, and the
-- voter_hash is never part of any view.
-- ------------------------------------------------------------------

-- Per course: the money-grab meter, the AI slop meter and the rest.
create or replace view public.course_review_summary
with (security_invoker = true) as
select
  course_id,
  count(*)                                          as reviews,
  count(*) filter (where verdict = 'worth_it')      as worth_it,
  count(*) filter (where verdict = 'money_grab')    as money_grab,
  round(100.0 * count(*) filter (where verdict = 'worth_it') / count(*)) as worth_it_pct,
  round(avg(ai_slop), 1)                            as avg_ai_slop,
  count(ai_slop)                                    as ai_slop_ratings,
  round(avg(learned), 1)                            as avg_learned,
  round(avg(difficulty), 1)                         as avg_difficulty,
  count(*) filter (where took_course = 'dropped')   as dropped,
  count(comment)                                    as comments,
  max(updated_at)                                   as last_review_at
from public.course_reviews
where status = 'published'
group by course_id;

-- The same per platform (Udemy, Coursera, ...).
create or replace view public.platform_review_summary
with (security_invoker = true) as
select
  platform,
  count(*)                                          as reviews,
  count(*) filter (where verdict = 'worth_it')      as worth_it,
  count(*) filter (where verdict = 'money_grab')    as money_grab,
  round(100.0 * count(*) filter (where verdict = 'worth_it') / count(*)) as worth_it_pct,
  round(avg(ai_slop), 1)                            as avg_ai_slop,
  round(avg(learned), 1)                            as avg_learned,
  count(*) filter (where took_course = 'dropped')   as dropped
from public.course_reviews
where status = 'published'
group by platform;

-- How often each tag was picked for each course ("Top tags").
create or replace view public.course_tag_counts
with (security_invoker = true) as
select r.course_id, t.code as tag, t.label, t.kind, count(*) as uses
from public.course_review_tags rt
join public.course_reviews r on r.id = rt.review_id and r.status = 'published'
join public.review_tags t on t.code = rt.tag
group by r.course_id, t.code, t.label, t.kind;

-- The reviews themselves, with helpful counts and tags.
create or replace view public.course_review_feed
with (security_invoker = true) as
select
  r.id, r.course_id, r.platform, r.took_course, r.verdict,
  r.ai_slop, r.learned, r.difficulty, r.paid, r.comment,
  r.created_at, r.updated_at,
  h.helpful, h.not_helpful,
  coalesce(tg.tags, '{}') as tags
from public.course_reviews r
left join lateral (
  select count(*) filter (where v.helpful)     as helpful,
         count(*) filter (where not v.helpful) as not_helpful
  from public.review_helpful_votes v
  where v.review_id = r.id
) h on true
left join lateral (
  select array_agg(t.tag order by t.tag) as tags
  from public.course_review_tags t
  where t.review_id = r.id
) tg on true
```

```
where r.status = 'published';


-- ------------------------------------------------------------------
-- Lock it down. The public key can't read or write anything here; only
-- the website's server, using the project's secret key, can. Newer
-- Supabase projects no longer give new tables API access by default,
-- so the secret key (service_role) gets exactly what it needs.
-- ------------------------------------------------------------------
alter table public.course_reviews       enable row level security;
alter table public.review_tags          enable row level security;
alter table public.course_review_tags   enable row level security;
alter table public.review_helpful_votes enable row level security;
alter table public.review_reports       enable row level security;

revoke all on
  public.course_reviews, public.review_tags, public.course_review_tags,
  public.review_helpful_votes, public.review_reports,
  public.course_review_summary, public.platform_review_summary,
  public.course_tag_counts, public.course_review_feed
from anon, authenticated;

revoke all on function public.hide_reported_review() from public, anon, authenticated;
revoke all on function public.submit_review(
  bigint, text, text, text, text, integer, integer, integer, text, text, text[]
) from public, anon, authenticated;

grant select, insert, update, delete on public.course_reviews       to service_role;
grant select                         on public.review_tags          to service_role;
grant select, insert, delete         on public.course_review_tags   to service_role;
grant select, insert, update, delete on public.review_helpful_votes to service_role;
grant select, insert                 on public.review_reports       to service_role;
grant select on
  public.course_review_summary, public.platform_review_summary,
  public.course_tag_counts, public.course_review_feed
to service_role;
grant execute on function public.submit_review(
  bigint, text, text, text, text, integer, integer, integer, text, text, text[]
) to service_role;

-- Tell the API to pick up the changes right away.
notify pgrst, 'reload schema';
```